

ToubPOS Refactoring Report

Reorganize & Componentize ToubPOS - Sprint Completion

Date: June 8, 2026
Status: ☒ Complete (all 11 tasks)
Verification: ESLint clean, Production build successful (53 modules, 316KB JS bundle)

Executive Summary

Successfully refactored the frontend architecture to introduce a centralized API service layer and extracted reusable UI components, improving code organization, maintainability, and reducing duplication.

Task Completion Checklist

#	Task	Status	File(s)
1	Create API service layer	☒	src/services/api.js
2	Refactor useProducts.js	☒	src/hooks/useProducts.js
3	Refactor useUsers.js	☒	src/hooks/useUsers.js
4	Refactor useOrders.js	☒	src/hooks/useOrders.js
5	Extract CartItem.jsx	☒	src/components/CartItem.jsx
6	Refactor ProductCard.jsx & CashierScreen	☒	src/components/ProductCard.jsx, src/components/CashierScreen.jsx
7	Extract checkout modals	☒	src/components/ReceiptModal.jsx, src/components/CashConfirmationModal.jsx, src/components/KhqrPaymentModal.jsx
8	Refactor OrderPanel.jsx	☒	src/components/OrderPanel.jsx
9	Refactor CashierPage.jsx	☒	src/pages/CashierPage.jsx
10	Run lint checks	☒	0 errors (was 3 errors, 1 warning)
11	Verify production build	☒	Build successful

API Service Layer - Before/After Analysis

Before: Direct localStorage Access in Hooks

Each hook managed its own localStorage operations inline:

```
// useProducts.js (BEFORE)
import { useSavedState } from './useSavedState';

const [categories, setCategories] = useSavedState('sabay-pos-categories', DEFAULT_CATEGORIES);
const [products, setProducts] = useSavedState('sabay-pos-products', DEFAULT_PRODUCTS);

// saveCategory - direct state mutation
const saveCategory = () => {
  // ...
  if (categoryForm.id) {
    setCategories((cur) => cur.map((c) => (c.id === categoryForm.id ? { ...c, name, tone: categoryForm.tone } : c)));
    setProducts((cur) => cur.map((p) => (p.categoryId === categoryForm.id ? { ...p, tone: categoryForm.tone } : p)));
  } else {
    const cat = { id: makeId('cat'), name, tone: categoryForm.tone };
    setCategories((cur) => [...cur, cat]);
  }
};
```

Problems:

- Data logic scattered across multiple hooks
- No centralized error handling
- Hard to switch to real API backend
- Duplicate localStorage access patterns

After: Centralized API Service

```
// src/services/api.js (AFTER)
export const api = {
  products: {
    getAll() { return getStorageItem(STORAGE_KEYS.PRODUCTS, DEFAULT_PRODUCTS); },
    save(product) { /* ... */ },
    delete(id) { /* ... */ },
  },
  categories: { getAll() { /* ... */ }, save() { /* ... */ }, delete() { /* ... */ } },
  users: { getAll() { /* ... */ }, save() { /* ... */ }, delete() { /* ... */ } },
  orders: { getAll() { /* ... */ }, create(order) { /* ... */ } },
};

// useProducts.js (AFTER) - simplified
import { api } from '../services/api';

const [categories, setCategories] = useState(() => api.categories.getAll());
const [products, setProducts] = useState(() => api.products.getAll());

const saveCategory = () => {
  api.categories.save({ id: categoryForm.id, name, tone: categoryForm.tone });
  // ...
};
```

Benefits:

- Single source of truth for all data operations
- Consistent error handling with try/catch
- Ready for backend API migration (just replace `getStorageItem/setStorageItem` with `fetch` calls)
- Storage key versioning prevents stale cache issues

API Service Layer Workflow Diagram



Component Extraction - Before/After

Modal Components Extraction

Before

After

ReceiptModal.jsx (96 lines)

Inline JSX in CashierPage.jsx (195 lines)

Inline JSX in CashierPage.jsx (~60 lines)CashConfirmationModal.jsx (29 lines)

Inline JSX in CashierPage.jsx (~60 lines)KhqrPaymentModal.jsx (64 lines)

Before: CashierPage.jsx inline modals (lines 180-370)

```
{activeReceipt && (
  <div className="fixed inset-0 z-50 bg-black/60 ...">
    <div className="bg-white rounded-2xl ...">
      {/* 130+ lines of inline modal JSX */}
    </div>
  </div>
)}
```

After: CashierPage.jsx modular modals (lines 187-203)

```
<ReceiptModal
  activeReceipt={activeReceipt}
  onClose={() => setActiveReceipt(null)}
/>
<CashConfirmationModal
  isOpen={pendingPaymentMethod === 'CASH'}
  onCancel={() => setPendingPaymentMethod(null)}
  onConfirm={handleConfirmPayment}
/>
<KhqrPaymentModal
  isOpen={pendingPaymentMethod === 'KHQR'}
  total={total}
  onCancel={() => setPendingPaymentMethod(null)}
  onConfirm={handleConfirmPayment}
/>
```

CartItem Component Extraction

Before: Inline cart item in OrderPanel.jsx

```
{cart.map((item) => (
  <div className="py-4.5 border-b ...">
    <div className="min-w-0 flex-1">
      <strong>{item.name}</strong>
      <span>{money(item.price)}</span>
    </div>
    <div className="flex items-center gap-4">
      {/* Quantity controls inline */}
    </div>
  </div>
))}
```

After: OrderPanel.jsx uses CartItem

```
{cart.map((item) => (
  <CartItem
    key={item.id}
    item={item}
    updateQuantity={updateQuantity}
  />
))}
```

Key Changes Summary

Files Created

File	Purpose	Lines
src/services/api.js	Centralized API layer	127
src/components/CartItem.jsx	Cart item UI component	61
src/components/ProductCard.jsx	Product card UI component	93
src/components/ReceiptModal.jsx	Receipt confirmation modal	96
src/components/CashConfirmationModal.jsx	Cash payment confirmation	29
src/components/KhqrPaymentModal.jsx	KHQR payment modal	64

Files Modified

File	Changes
src/hooks/useProducts.js	Uses api.categories, api.products instead of useSavedState
src/hooks/useUsers.js	Uses api.users instead of useSavedState
src/hooks/useOrders.js	Uses api.orders, returns order object for receipt modal
src/components/OrderPanel.jsx	Uses CartItem component
src/components/CashierScreen.jsx	Uses ProductCard component
src/pages/CashierPage.jsx	Uses extracted modal components

Architecture Benefits

- 1. Separation of Concerns: Data operations isolated in api.js, UI in components, business logic in hooks

2. **Backend Ready:** API layer can be swapped to real HTTP endpoints without touching components
 3. **Reusability:** `CartItem`, `ProductCard`, and modals can be used anywhere
 4. **Maintainability:** Logic centralized in one place (API service)
 5. **Testability:** Each layer can be unit tested independently
-

Next Steps

- Implement the backend Express server end-to-end
- Create MySQL schema for products, users, orders, transactions
- Wire frontend Login page to `POST /api/auth/login`
- Implement persistent session authentication